

NIRIKSHAK (■■■■■■■■■■) — Engineering Status Report

Complete Technical Implementation Dossier: Foundation, ML, Correlation, Console, Simulator & QA

Framework: NIRIKSHAK (Decision Support Engine)	Build Status: 100% OPERATIONAL (18/18 Tests Green)
Stack: FastAPI (Python 3.11) + Next.js 14 + PostgreSQL 16	Live Dashboard: http://localhost:3001
Axiom: Unusual Activity ≠ Confirmed Malicious Activity	Scope: All 4 Engineering Tracks Fully Implemented

1. Executive Summary & Foundational Axiom

NIRIKSHAK (■■■■■■■■■■ — Sanskrit for 'The Inspector / Observer') is a defensive cybersecurity and insider-risk decision-support platform. It solves a pervasive operational vulnerability in modern SOC environments: **authenticated users abusing legitimate access privileges**.

CORE DEFENSIVE PHILOSOPHY

The Foundational Axiom: Unusual Activity ≠ Confirmed Malicious Activity.

NIRIKSHAK is strictly a **Decision Support System**, never an autonomous disciplinary or accusation engine. Its operational goal is to continuously evaluate whether access to sensitive resources remains contextually reasonable, even when a user is technically authenticated and authorized. All high-risk decisions strictly require human security analyst review.

2. System Architecture: The Seven Conceptual Layers

Every telemetry event ingested by NIRIKSHAK traverses seven defined analytical layers before reaching the analyst dashboard:

Layer	Layer Name	Analytical Function & Scope	Weight in Model
1	Identity Confidence	Evaluates authentication state, MFA freshness, credential velocity.	23% (w_I)
2	Device Trust	Hardware fingerprint, trusted hardware inventory, revocation status.	23% (w_D)
3	Data Sensitivity	Target resource classification (LOW, INTERNAL, CONFIDENTIAL, CRITICAL).	23% (w_S)
4	Behavioral Context	Statistical deviation vs. historical user and department baselines.	23% (w_B)
5	Anomaly Detection (ML)	Unsupervised Isolation Forest 6D vector density scoring.	8% (w_ML)
6	Event Correlation	15-minute sliding temporal window; reconnaissance & exfiltration chaining.	Multiplicative
7	Human Review	Security analyst case workflow, disposition actions, immutable audit trail.	Governance

Strict 4-Tier Backend Architecture:

To prevent spaghetti architecture, the backend strictly enforces separation of concerns: API Layer (FastAPI Routers) → Service Layer (Business Logic Orchestration) → Engines / Repository (Scoring & DB Queries) → PostgreSQL 16 Database.

3. Track 1: Dynamic Policies & Device Trust Admin

In enterprise defense, risk weights and device trust states cannot be hard-coded. Track 1 provides real-time policy and hardware trust controls:

- **Dynamic Policy Service (backend/app/services/policy_service.py):** Manages configurable weights (WEIGHT_IDENTITY: 0.23, WEIGHT_DEVICE: 0.23, WEIGHT_SENSITIVITY: 0.23, WEIGHT_BEHAVIOR: 0.23, WEIGHT_ML_ANOMALY: 0.08) and operational thresholds (MODERATE: 31, HIGH: 61, CRITICAL: 81). Stored in the risk_policies database table with caching and fallback defaults.
- **Policy Management API (backend/app/api/policies.py):** Exposes GET /api/v1/policies and PUT /api/v1/policies/{key}. Enforces strict regex validation (^[A-Z0-9_]{3,40}\$) to eliminate command injection, automatically updates system policy, and writes an immutable audit record.
- **Device Trust Lifecycle API (backend/app/api/devices.py):** Enables security teams to inspect registered hardware (GET /api/v1/devices) and dynamically toggle trust states (PATCH /api/v1/devices/{id}/trust: TRUSTED, MONITORED, REVOKED). A REVOKED device immediately incurs a +90 point trust penalty on subsequent access attempts.
- **Immutable Audit Trail (backend/app/models/audit_log.py):** Every policy tweak, device revocation, and case disposition creates an append-only audit log recording action, actor_user_id, previous_state, and new_state.

4. Track 2: Isolation Forest Anomaly Detection & Temporal Correlation

Track 2 integrates unsupervised statistical machine learning and temporal multi-event chain analysis:

Component	Implementation Details	Operational Impact
Feature Pipeline ml/features/extractor.py	Extracts normalized 6D observation vector: [hour_deviation, volume_z_score, resource_sensitivity, device_familiarity, department_mismatch, action_severity].	Standardizes disparate telemetry into a unified mathematical representation.
Isolation Forest Model ml/training/train.py	IsolationForest(n_estimators=100, contamination=0.04) trained on 3,000 synthetic observations. Artifact: ml/models/isolation_forest_v1.joblib with metadata.	Identifies multi-dimensional geometric outliers without human bias.
ML Anomaly Engine backend/app/engines/anomaly_engine.py	Infers \$[0, 100]\$ score with factor attribution: ML_ISOLATION_FOREST_ANOMALY (outlier score) or ML_BEHAVIORAL_ALIGNMENT (inlier credit).	Acts strictly as an 8% additive signal; never convicts autonomously.
Correlation Engine backend/app/engines/correlation_engine.py	Evaluates 15-minute sliding temporal window. Detects repository hopping, rapid directory enumeration, and recon-to-exfiltration sequences.	Applies \$1.25\times - 1.35\times\$ multiplier when multi-stage attacks are correlated.

5. Track 3: Modern SOC Frontend, Recharts & Hybrid Command Console

Track 3 equips security analysts with dual visualization and control modalities: a modern graphical SOC dashboard and an integrated high-velocity analyst terminal.

1. Modern SOC Dashboard (frontend/app/page.tsx):

Built with Next.js 14, React, and Tailwind CSS. Features live KPI summary cards (Total Events, Active Cases, High Risk Rate, Blocked Threats), one-click interactive scenario triggers, real-time event feed, case investigation drawer, and factor contribution breakdown.

2. Longitudinal Risk-Velocity Visualization (frontend/components/analytics/RiskTrendChart.tsx):

Interactive Recharts area chart tracking risk scores chronologically. Renders visual threshold reference lines for MODERATE (31), HIGH (61), and CRITICAL (81). Displays risk velocity and acceleration spikes as multi-stage attacks unfold.

3. Hybrid Analyst Command Console (frontend/components/terminal/CommandConsole.tsx):

Accessible via docked drawer or global hotkey (Ctrl + ~). Designed for keyboard-first security triage. Features Tab autocomplete, command history buffer, and structured ASCII tables.

SECURITY ARCHITECTURE: TERMINAL IS NOT A BACKDOOR

Zero Browser OS Shell Enforcement:

The terminal is an application AST allowlist parser communicating directly with authenticated REST endpoints. It strictly rejects sh, bash, curl, sudo, or sub-processes.

Permitted Commands: help, status, events, cases, investigate <user_id>, explain <event_id>, devices, policy, clear.

6. Track 4: Multi-Stage Attack Simulator & Security Boundary QA

Track 4 provides end-to-end operational verification, automated adversarial testing, and hackathon presentation choreography:

1. Multi-Stage APT Kill-Chain Simulator (simulator/killchain_generator.py):

Simulates a realistic Advanced Persistent Threat (APT) insider scenario across 4 distinct phases:

- **Stage 1: External Reconnaissance** → Off-network IP accesses public docs (Score: 21.1 LOW).
- **Stage 2: Lateral Enumeration** → Rapid directory scanning across internal assets (Score: 35.8 MODERATE).
- **Stage 3: Privilege Bypass** → Direct access to classified operational repository without MFA (Score: 51.4 MODERATE).
- **Stage 4: Bulk Exfiltration** → 1.8 GB compressed archive export from untrusted device (Score: 74.4 HIGH) → Triggers Security Case.

2. Automated Security Boundary Testing (tests/security/test_security_boundaries.py):

Four automated security test suites verify non-negotiable boundaries:

- **SQL Injection Neutralization:** Verified parameterized queries reject ' OR '1'='1 payloads.
- **Command Injection Rejection:** Verified policy update API rejects shell metacharacters (; rm -rf /, | cat /etc/passwd).
- **Audit Log Immutability:** Verified audit logs cannot be updated or deleted via API.
- **Case Review RBAC:** Verified case disposition requires authenticated human security review.

Test Suite	Test Scope	Result
backend/app/tests/ (14 tests)	Models, Ingestion API, Risk Engines, Policy Service, Device API	14 PASSED
tests/security/ (4 tests)	SQL Injection, Shell Metacharacters, Audit Immutability, RBAC	4 PASSED
TOTAL	Complete System Integration & Security Boundary Suite	18 / 18 PASSED (100%)

3. Choreographed 5-Minute Presentation Script (docs/DEMO_SCRIPT.md):

A turn-by-turn presentation script allocating distinct speaking roles to all 4 team members: **Speaker 1** (The Problem & Axiom), **Speaker 2** (Live Telemetry Ingestion & Scoring), **Speaker 3** (APT Kill-Chain Simulation & Temporal Correlation), and **Speaker 4** (Command Console & Case Resolution). Includes prepared defenses for 5 tough judge Q&A; questions.

7. Codebase Structure & File Manifest

The repository follows a clean, modular layout with zero circular dependencies:

Directory / File Path	Description & Architectural Purpose
backend/app/main.py	FastAPI application entrypoint, CORS configuration, route registration.
backend/app/core/database.py	Async SQLAlchemy engine, PostgreSQL connection pool, schema auto-patching.
backend/app/models/	SQLAlchemy entities: User, Device, Resource, Event, RiskScore, Case, AuditLog, Policy.
backend/app/engines/risk_engine.py	Composite multi-layer scoring engine with deterministic factor attribution.
backend/app/engines/anomaly_engine.py	Isolation Forest runtime inference and feature importance breakdown.
backend/app/engines/correlation_engine.py	15-minute sliding-window temporal chain and kill-chain correlation.
backend/app/api/policies.py	Dynamic risk weight and operational threshold admin endpoints.
backend/app/api/devices.py	Hardware inventory and trust state transition endpoints.
backend/app/api/scenarios.py	One-click scenario injection API (Normal, Off-Hours, Exfiltration, Kill Chain).
ml/features/extractor.py	6D observation vector extraction from raw canonical telemetry.
ml/training/train.py	Isolation Forest training pipeline producing versioned model artifacts.
frontend/app/page.tsx	Main SOC analyst dashboard, KPI statistics, interactive scenarios.
frontend/components/terminal/CommandConsole.tsx	Docked analyst command console with strict AST allowlist parsing.
frontend/components/analytics/RiskTrendChart.tsx	Recharts longitudinal risk velocity visualization.
simulator/killchain_generator.py	Standalone CLI tool generating 4-stage APT attacks against the API.
tests/security/test_security_boundaries.py	Automated security test suite verifying SQL injection, shell metacharacters, RBAC.
docs/DEMO_SCRIPT.md	Turn-by-turn 5-minute presentation script for team of 4.
docs/TEAM_ALLOCATION.md	Division of labor document mapping each member's contributions.

8. Deployment Status & Verification Logs

All services are containerized via Docker Compose and verified healthy:

Service Container	Internal Port	Host Port	Health Status
nirikshak-db (PostgreSQL 16.3)	5432	localhost:5432	HEALTHY
nirikshak-backend (FastAPI / Uvicorn)	8000	localhost:8000	HEALTHY (Rebuilt w/ ML)
nirikshak-frontend (Next.js 14)	3000	localhost:3001	HEALTHY (TypeScript 0 err)

Port Conflict Resolution Note:

Due to an existing macOS background Node process on port 3000, docker-compose.yml was configured to map the frontend container to host port **3001** (<http://localhost:3001>) and backend CORS origins were updated accordingly. Database schema auto-patching ensures all required tables and columns (including ML and correlation attributes) initialize automatically on startup.

NIRIKSHAK Technical Dossier — End of Engineering Report